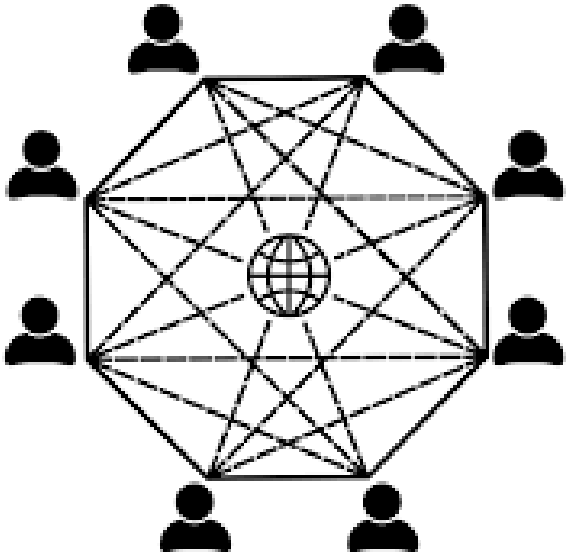
네트워크설계

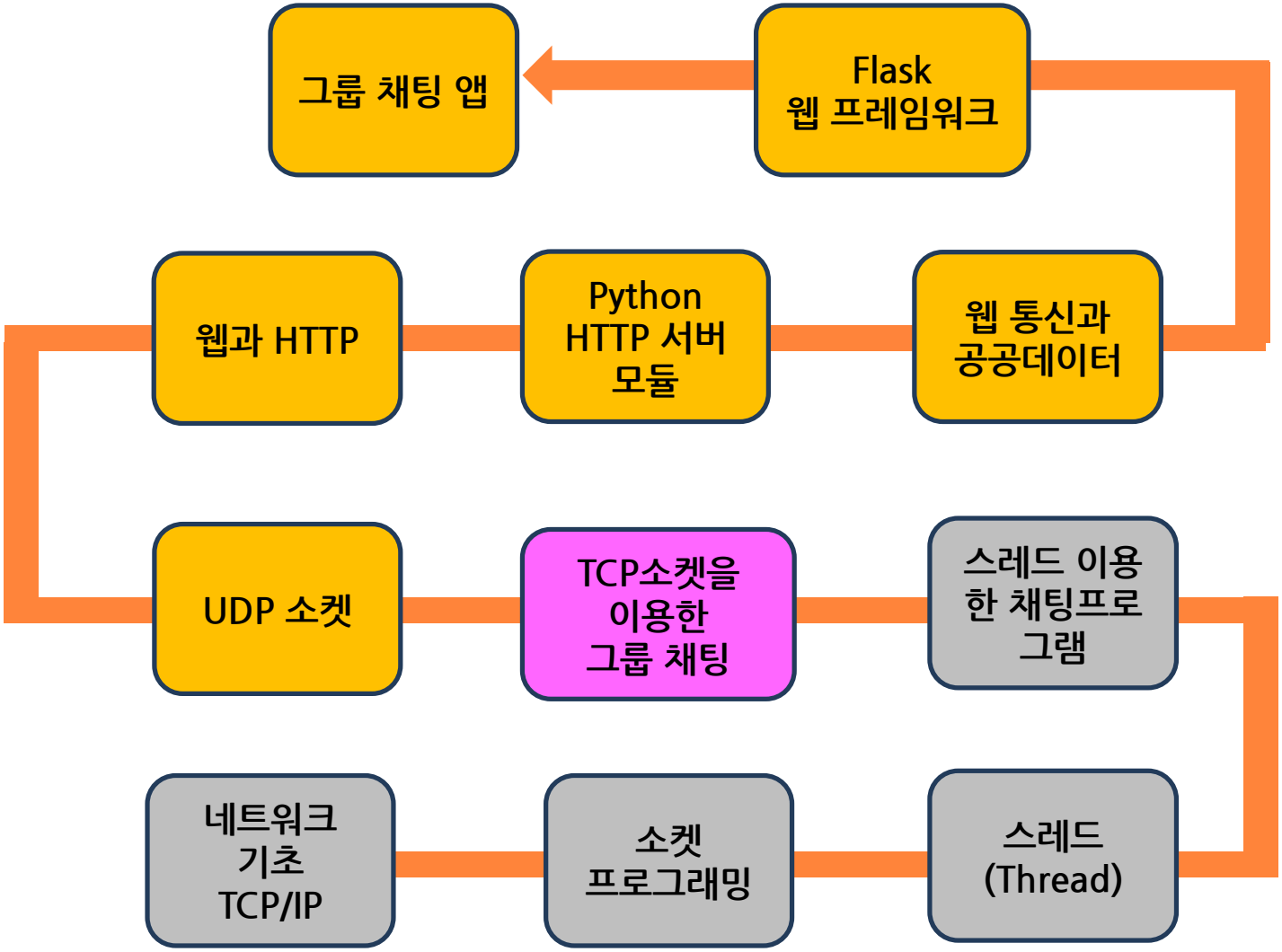
5주차 - 스레드와 TCP 소켓을 이용한 그룹 채팅



인공지능학과

그림참조: https://pxhere.com/ko/photo/1637436

5주차는 TCP 소켓을 이용한 그룹채팅



앱 제작 실습 병행

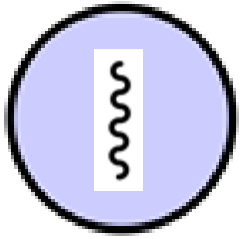
4주차 정리. 스레드 개념

■ 프로세서와 스레드

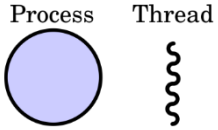
- 프로세스(Process): 실행중인 프로그램
- 스레드(Thread): 프로세스 안에서 병렬로 실행중인 하위 작업 단위
- 기본적으로 파이썬은

  프로세스는 기본적으로 하나의 싱글 스레드에서 동작 : 메인 스레드(main thread)

  메인 스레드가 코드를 순차적으로 실행



싱글 스레드
(메인 스레드)



4주차 정리. 스레드 생성과 실행

- **스레드 사용법**
  - **threading.Thread( ) 함수 호출 방식**
  - threading 모듈의 threading.Thread( ) 이용하여 thread 객체 생성 후 start( ) 호출방식

```
# Thread 사용하기

import threading, time

def send(n):
    for i in range(n):
        print("send msg:", i)
        time.sleep(1)

def recv(n):
    for i in range(n):
        print("recv msg:", i)
        time.sleep(1)

if __name__ == "__main__":
    # 스레드 객체 생성하고 실행하기
    sender = threading.Thread(target=send, args=(5,))
    sender.start()
    receiver = threading.Thread(target=recv, args=(5,))
    receiver.start()
```

서브 스레드

서브 스레드

메인 스레드

```
send msg: 0
recv msg: 0
recv msg: 1
send msg: 1
recv msg: 2
send msg: 2
recv msg: 3
send msg: 3
recv msg: 4
send msg: 4
```

# 스레드 객체 생성하기 (매개변수가 있는 경우)
threading.Thread(target=함수, args=(함수의 매개변수,))
args는 튜플만 지원하며, 하나의 매개변수를 전달하는 경우, 쉼표(,)를 추가해 튜플로 설정

스레드 생성
스레드 실행

4주차 정리. join( ) 사용법

- **메인 스레드가 스레드 종료까지 기다리려면?**
  - 서브 스레드가 종료될 때까지 기다린 후 메인 스레드를 실행하려면 스레드 join( ) 사용
  - **join( ): 다른 스레드의 종료를 기다린 후 실행**

```python
# 스레드 생명주기

import threading, time

count = 0

def worker(n):
    global count
    print("worker thread start")
    for i in range(n):
        count += 1
        time.sleep(1)
    print("worker thread end")

if __name__ == "__main__":
    # 스레드 객체 생성 및 실행
    t = threading.Thread(target=worker, args=(5,))
    t.start()
    # 스레드 종료시까지 대기
    t.join()

    # count 출력
    print("count:", count)
```

```
worker thread start
worker thread end
count: 5
```

서브 스레드 작업 종료 후
메인 스레드 실행

4주차 정리. Event(이벤트) 객체 사용법

■ 스레드 실습3: 이벤트 객체를 사용해 서브 스레드 종료 시키기

- threading 모듈의 Event 객체를 사용
- Event 객체의 set( )을 호출하면 객체의 내부 플래그 값을 True로 설정
- is_set( )을 사용하여 플래그 값을 확인 후 서브 스레드를 종료

```python
# 무한루프 스레드 종료하기 - 이벤트 객체 사용

import threading, time

#이벤트 객체 생성
event = threading.Event()

def loop():
    while True:
        time.sleep(1)
        if event.is_set(): # 이벤트 객체 flag 확인
            print("Infinite Loop thread stop!")
            return
        print("Infinite Loop thread!")

if __name__ == "__main__":
    # 스레드 객체생성 후 실행하기
    t = threading.Thread(target=loop)
    t.start()

    print("main thread start")
    time.sleep(5)
    event.set()   # 이벤트 객체 flag set
    print("main thread end")
```

```
main thread start
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
main thread end
Infinite Loop thread stop!
```

메인 스레드 종료 시 서브 스레드 종료

4주차 정리. 스레드간 통신 – 큐 사용

- **과제: 토끼와 거북이의 경주에서 누가 이겼는지 스레드 사용해서 코드로 구현하기**

경주 길이는 총 30m
- 거북이는 1초에 1m의 속도,
- 토끼는 0.5초에 1m의 속도로 달린다.
  토끼는 25m 지점에서 낮잠을 자고 17초 후에 깨어난다
  낮잠에서 깨어난 토끼는 나머지 구간을 0.3초에 1m 속도로 달린다
- 경주가 완료된 후 최종 승자를 출력한다

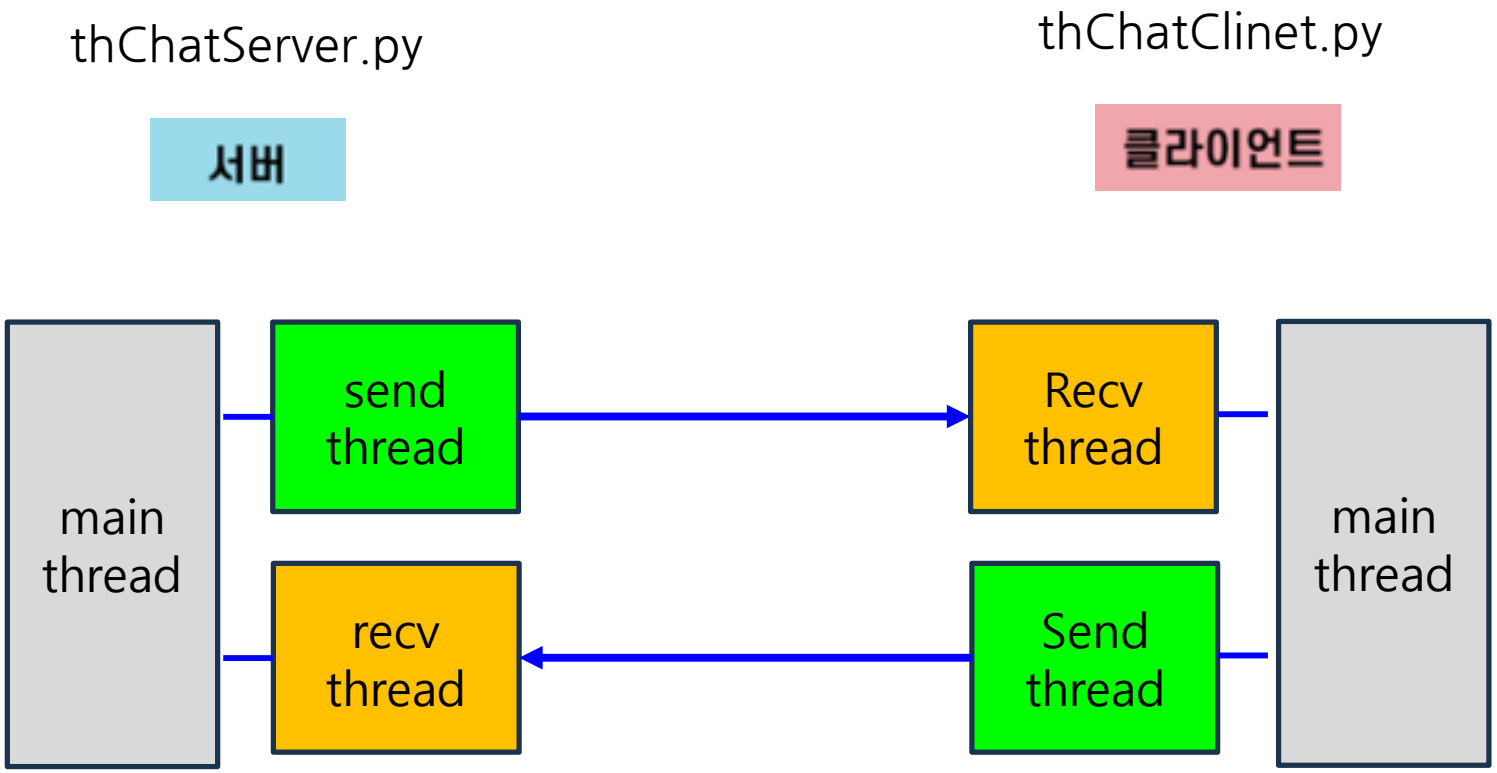
위 조건인 경우 최종 승자는 누구인지 코드로 확인하기
단, 달린 거리는 1m 단위로 출력 (예: 거북이-1m, 토끼-8m)

그림출처: rajathathablog. the rabbit vs the tortoise

4주차 정리. 스레드와 TCP 소켓을 이용한 1:1 채팅

■ 스레드와 TCP 소켓을 이용해 1:1 채팅하기

- 스레드를 이용해 언제든 메시지를 주고 받을 수 있도록 채팅 프로그램을 구현해 봅시다



4주차 정리. 스레드와 TCP 소켓을 이용한 1:1 채팅 - 서버

- 스레드와 TCP 소켓을 이용한 1:1 채팅 서버 프로그램 (파일: th_tcpChatServer.py)

```python
# TCP chatting server program w/ thread

import socket, threading

def recv(socket):
    while True:
        rxmsg = socket.recv(1024).decode()
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Client, please 'exit'...")
            break
        print("Client >>", rxmsg)

def send(socket):
    while True:
        sendmsg = input("Server >> ")
        socket.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print("Exit by User")
            break

if __name__ == "__main__":
    IP = 'localhost'  # loopback ip (127.0.0.1)
    PORT = 9999
    address = (IP, PORT)  # tuple로 주소정보 저장

    print(">> Server start")
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # (optional code) winError 10048 에러 해결을 위한 추가 코드
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)
    server.bind(address)
    server.listen()
    print(">> Server wait.........")
    conn, addr = server.accept()
    print(f">> {addr} is connected")
    # 메시지 송신과 수신을 스레드로 생성하고 실행
    receiver = threading.Thread(target=recv, args=(conn,))
    receiver.start()
    sender = threading.Thread(target=send, args=(conn,))
    sender.start()
    while True:
        receiver.join()
        sender.join()
        break
    conn.close()      # 통신소켓 종료
    server.close()       # 리스닝소켓 종료
    print(">> Server End")
```

4주차 정리. 스레드와 TCP 소켓을 이용한 1:1 채팅 - 클라이언트

- 스레드와 TCP 소켓을 이용한 1:1 채팅 클라이언트 프로그램(파일: th_tcpChatClinet.py)

```python
# TCP chatting client program w/ thread

import socket, threading

def send(socket):
    while True:
        sendmsg = input("Client >>")
        socket.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print(">> Exit by User")
            break

def recv(socket):
    while True:
        rxmsg = socket.recv(1024).decode('utf-8')
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Server, please 'exit'...")
            break
        print("Sever >>", rxmsg)

if __name__ == "__main__":
    IP = 'localhost'  # 서버 IP
    PORT = 9999       # 서버 포트번호
    address = (IP, PORT)  # tuple로 주소정보 저장

    print(">> Client start")
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(address)
    print(">> Connect server")
    # 메시지 송신과 수신을 스레드로 생성하고 실행
    receiver = threading.Thread(target=recv, args=(client,))
    receiver.start()
    sender = threading.Thread(target=send, args=(client,))
    sender.start()

    while True:
        receiver.join()
        sender.join()
        break

    client.close()
    print("Client End")
```

4주차 정리. 스레드와 TCP 소켓을 이용한 1:1 채팅

■ 실습 시 네트워크 exception 발생 사유

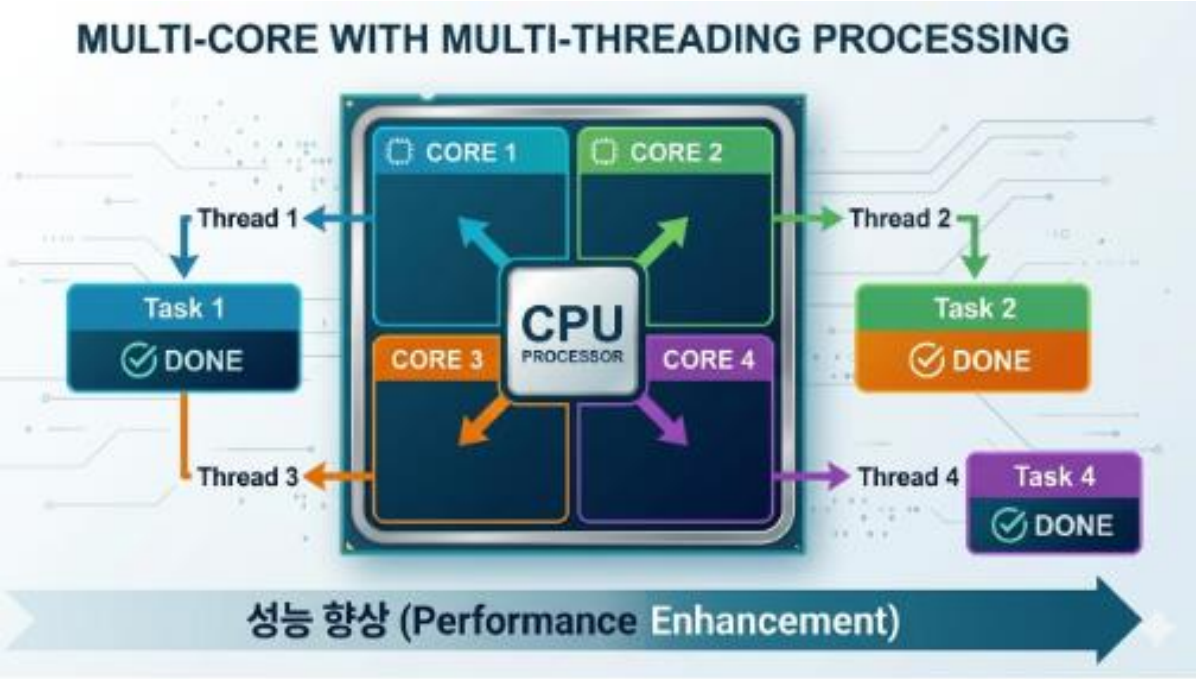
- 실습 시 가장 많이 발생하는 에러 유형은 WinError 10053
- 기존 TCP 연결을 강제로 종료했을 때 발생하는 Windows 소켓 오류
- 이미 소켓이 종료되었는데, send( ) 혹은 recv( )하려고 할 때 발생하는 오류

```
Exception in thread Thread-3 (receive_messages):
Traceback (most recent call last):
  File "C:\Users\User\anaconda3\Lib\threading.py", line 1043, in _bootstrap_inner
    self.run()
    ~~~~~~~~^^
  File "C:\Users\User\anaconda3\Lib\site-packages\ipykernel\ipkernel.py", line 766, in run_closure
    _threading_Thread_run(self)
    ~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\User\anaconda3\Lib\threading.py", line 994, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Public\Documents\ESTsoft\CreatorTemp\ipykernel_16240\82301631.py", line 9, in receive_messag
    data = sock.recv(1024).decode('utf-8')
           ~~~~~~~~~~^^^^^^
ConnectionAbortedError: [WinError 10053] 현재 연결은 사용자의 호스트 시스템의 소프트웨어의 의해 중단되었습니다
```

- 서버 IP주소를 'localhost'를 사용할 때와 실제 IP를 사용할 때는 패킷의 경로가 다름
  예) 실제 IP사용시 - LAN카드←→ 스위치 ←→ 라우터 ←→ 방화벽 ←→ 외부망
  따라서 에러 유형이 다르게 나타날 수 있음

1. 파이썬 멀티스레드와 성능

- 파이썬 멀티 스레드와 성능에 대해 알아봅시다



멀티 스레드를 사용해서
여러 작업을 동시에 실행하여 처리 속도를 향상시키며,
작업을 병렬로 처리함으로써 시간을 절약할 수 있음.
단, 파이썬도 그럴까?

1. 파이썬 멀티스레드와 성능

■ 파이썬 멀티스레드를 사용하면 성능이 향상될까?

단일 서브 스레드로 연산 시 소요시간

```python
# 파이썬 스레드와 성능

import threading, time

def total_sum(start, end, sumList):
    current_thread = threading.current_thread().name
    print(f"{current_thread} start")
    result = 0
    for i in range(start, end):
        result += i
    sumList.append(result)
    print(f"{current_thread} end")
    return

if __name__=="__main__":
    start_time = time.time()  # 시작시간
    numbers = list()
    start = 0
    end = 100000000
    t = threading.Thread(target=total_sum, args=(start, end+1,numbers))
    t.start()
    t.join()
    print(f"결과: {sum(numbers)}, 소요시간:", time.time()-start_time)
```

```
Thread-5 (total_sum) start
Thread-5 (total_sum) end
결과: 5000000050000000, 소요시간: 4.622211456298828
```

▪ 멀티스레드 사용시 성능이 향상될까?

멀티 스레드로 연산 시 소요시간

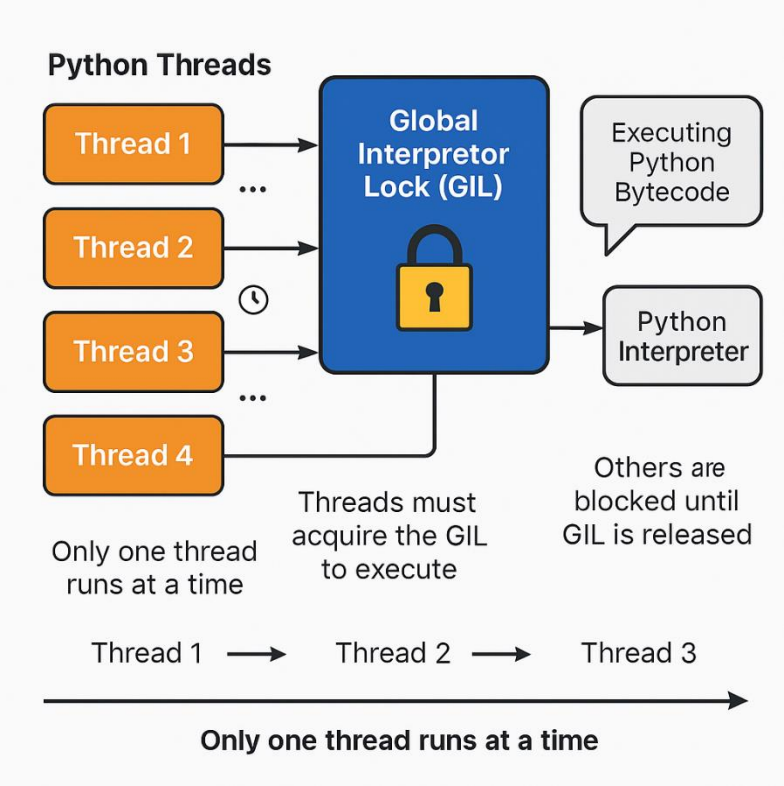
```python
# 파이썬 스레드와 성능

import threading, time

def total_sum(start, end, sumList):
    current_thread = threading.current_threa
    print(f"{current_thread} start")
    result = 0
    for i in range(start, end):
        result += i
    sumList.append(result)
    print(f"{current_thread} end")
    return

if __name__=="__main__":
    start_time = time.time()  # 시작시간
    numbers = list()
    start = 0
    end = 100000000
    t1 = threading.Thread(target=total_sum, args=(start, end//2,numbers))
    t2 = threading.Thread(target=total_sum, args=(end//2, end+1,numbers))
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    t.join()
    print(f"결과: {sum(numbers)}, 소요시간:", time.time()-start_time)
```

```
Thread-13 (total_sum) start
Thread-14 (total_sum) start
Thread-13 (total_sum) end
Thread-14 (total_sum) end
결과: 5000000050000000, 소요시간: 4.948898553848267
```

왜 멀티 스레드를 사용했는데 성능향상이 안될까?

파이썬의 GIL Global Interpreter Lock

파이썬 멀티스레딩은 **GIL(Global Interpreter Lock)**로 인해
CPU 집중 작업(CPU-bound)에서는 성능 향상이 거의 없거나 오히려 느려질 수 있음
하지만 네트워크 요청, 파일 I/O 등 대기 시간이 긴 작업(I/O-bound)에서는 스레드 간 전환을 통해
병렬 처리 효과를 내어 성능이 향상

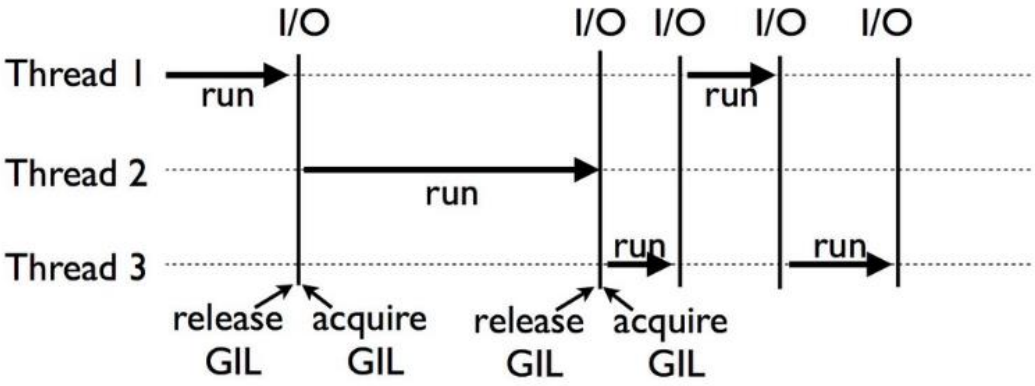


그림출처: kubeblogs.com

2. 파이썬 멀티스레드와 성능 - 파이썬 GIL

파이썬의 GIL Global Interpreter Lock

- 파이썬 GIL이란,
  여러 개의 스레드 중 하나의 스레드만 실행될 수 있도록 하는 **mutex(상호배제)**다



- 파이썬의 garbage collection 메모리 관리 정책을 사용하기 때문

- 단, 스레드가 I/O 작업을 실행하는 동안에는 다른 스레드가 CPU 동작을 동시에 실행할 수 있음
  I/O동작이 많은 프로그램에서는 멀티 스레드의 효과를 얻을 수 있음
  예) 네트워크 통신

2. 파이썬 멀티스레드와 성능 – CPU-bound job 인 경우

- CPU_bound job 인 경우

```python
# 파이썬 스레드의 성능과 GIL(Global Interpreter Lock:잠금장치)
import threading, time

# CPU_bound job : CPU 연산이 대부분인 job (CPU 활용률이 많음)
def cpu_bound(n):
    current_thread = threading.current_thread().name
    print(f"{current_thread} cpu_bound job start")
    while n:
        n -= 1
    print(f"{current_thread} cpu_bound job end")

if __name__ == "__main__":
    start = time.time()
    n = 200000000
    cpu_bound(n)
    cpu_bound(n)
    print("cpu_bound 소요시간:", time.time()-start)
```

```
MainThread cpu_bound job start
MainThread cpu_bound job end
MainThread cpu_bound job start
MainThread cpu_bound job end
cpu_bound 소요시간: 12.971206903457642
```

2. 파이썬 멀티스레드와 성능 – CPU-bound job 인 경우

▪ CPU-bound job 인 경우

```python
# 파이썬 스레드의 성능과 GIL(Global Interpreter Lock:잠금장치)
import threading, time

# CPU_bound job : CPU연산이 대부분인 job (CPU 활용률이 많음)
def cpu_bound(n):
    current_thread = threading.current_thread().name
    print(f"{current_thread} cpu_bound job start")
    while n:
        n -= 1
    print(f"{current_thread} cpu_bound job end")

if __name__ == "__main__":
    start = time.time()
    n = 200000000
    t1 = threading.Thread(target=cpu_bound, args=(n,))
    t2 = threading.Thread(target=cpu_bound, args=(n,))
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    print("cpu_bound thread 소요시간:", time.time()-start)
```

```
Thread-5 (cpu_bound) cpu_bound job start
Thread-6 (cpu_bound) cpu_bound job start
Thread-5 (cpu_bound) cpu_bound job end
Thread-6 (cpu_bound) cpu_bound job end
cpu_bound thread 소요시간: 11.971778392791748
```

→ 파이썬 멀티 스레딩은 GIL(Global Interpreter Lock) 제약으로 인해 CPU 연산 작업(CPU-bound)에서는 성능 향상이 거의 없거나 오히려 떨어질 수 있음

2. 파이썬 멀티스레드와 성능 – I/O-bound job 인 경우

- I/O-bound job 인 경우

```python
# 파이썬 스레드의 성능과 GIL(Global Interpreter Lock:잠금장치)
import threading, time

# IO_bound job : IO 연산이 대부분인 job (CPU 활용률이 적음)
def io_bound(n):
    current_thread = threading.current_thread().name
    print(f"{current_thread} io_bound job start")
    time.sleep(n)
    print(f"{current_thread} io_bound job end")

if __name__ == "__main__":
    start = time.time()
    n = 5
    io_bound(n)
    io_bound(n)
    print("io_bound 소요시간:", time.time()-start)
```

```
MainThread io_bound job start
MainThread io_bound job end
MainThread io_bound job start
MainThread io_bound job end
io_bound 소요시간: 10.00167965888977
```

2. 파이썬 멀티스레드와 성능 – I/O-bound job 인 경우

- I/O-bound job 인 경우

```python
# 파이썬 스레드의 성능과 GIL(Global Interpreter Lock:잠금장치)
import threading, time

# IO_bound job : IO 연산이 대부분인 job (CPU 활용률이 적음)
def io_bound(n):
    current_thread = threading.current_thread().name
    print(f"{current_thread} io_bound job start")
    time.sleep(n)
    print(f"{current_thread} io_bound job end")

if __name__ == "__main__":
    start = time.time()
    n = 5
    t1 = threading.Thread(target=io_bound, args=(n,))
    t2 = threading.Thread(target=io_bound, args=(n,))
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    print("io_bound thread 소요시간:", time.time()-start)
```

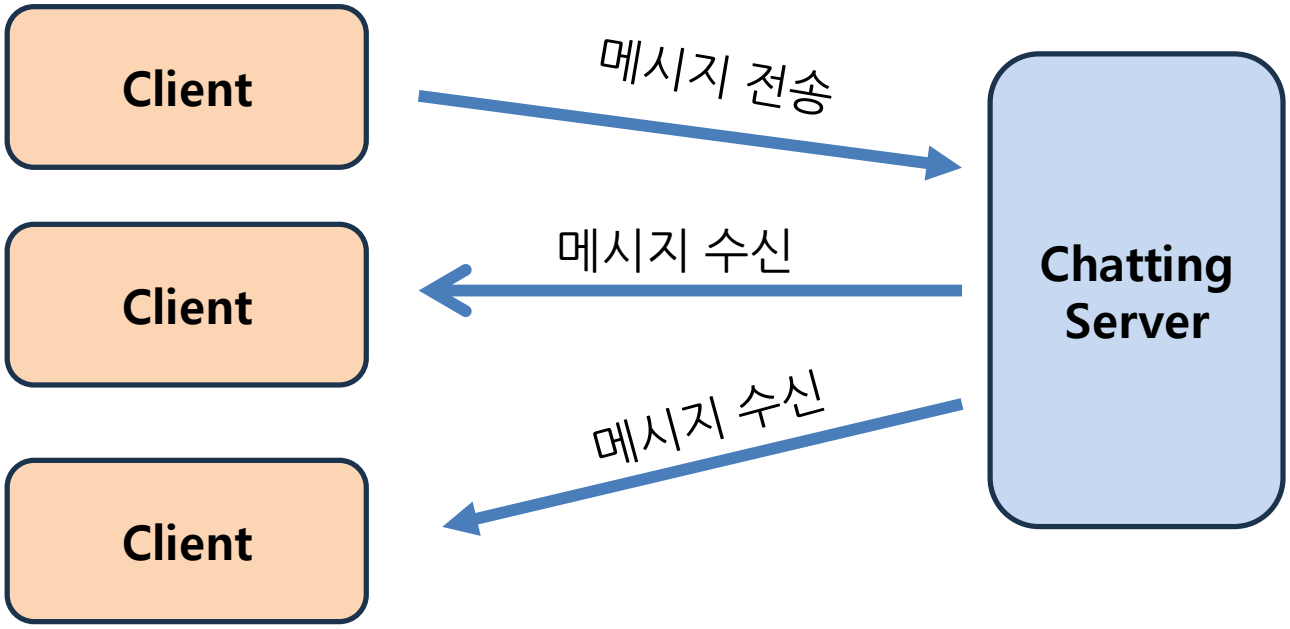
```
Thread-11 (io_bound) io_bound job start
Thread-12 (io_bound) io_bound job start
Thread-12 (io_bound) io_bound job end
Thread-11 (io_bound) io_bound job end
io_bound thread 소요시간: 5.0132410526275635
```

→ 파일 읽기/쓰기, 네트워크 요청 등 I/O 작업(I/O-bound)에서는 대기 시간 동안 다른 스레드를 실행해 효율을 높일 수 있음

3. TCP소켓과 스레드를 이용한 그룹채팅

TCP 소켓과 스레드를 이용해서 그룹 채팅 프로그래밍

- 그룹 채팅을 구현하기 위한 구성요소
  - 채팅 서버 : 클라이언트가 보낸 메시지를 연결된 모든 클라이언트에게 전송할 수 있는 서버
  - 다수 클라이언트들



▪ TCP 서버와 다수 클라이언트간 1:N 접속 방법

1:1 채팅 서버 프로그램 → 그룹 채팅

clients

클라이언트 소켓정보			

```
if __name__ == "__main__":
    IP = '10.2.40.25' #'127.0.0.1
    PORT = 9999      # 서버의 포트
    address = (IP, PORT)  # tuple

    print(">> Server start")
    server = socket.socket(socket
    # (필수코드아님) winError 1004
    server.setsockopt(socket.SOL_
    server.bind(address)
    server.listen()
    print(">> Server wait.......")

    conn, addr = server.accept()
    print(f">> {addr} is connected" )

    receiver = threading.Thread(target=Recv,args=(conn,))
    receiver.start()
    sender = threading.Thread(target=Send,args=(conn,))
    sender.start()
```

```
def client_handle():
    while True:
        try:
            client, addr = server.accept()
            print(f"{addr} is connected")
            # 클라이언트 소켓 추가
            clients.append(client)
            # 새로 연결 요청된 클라이언트의 수신 스레드 생성 및 시작
            receive = threading.Thread(target=recv, args=(client,))
            receive.start()
```

연결이 될 때마다 클라이언트 소켓 정보 저장

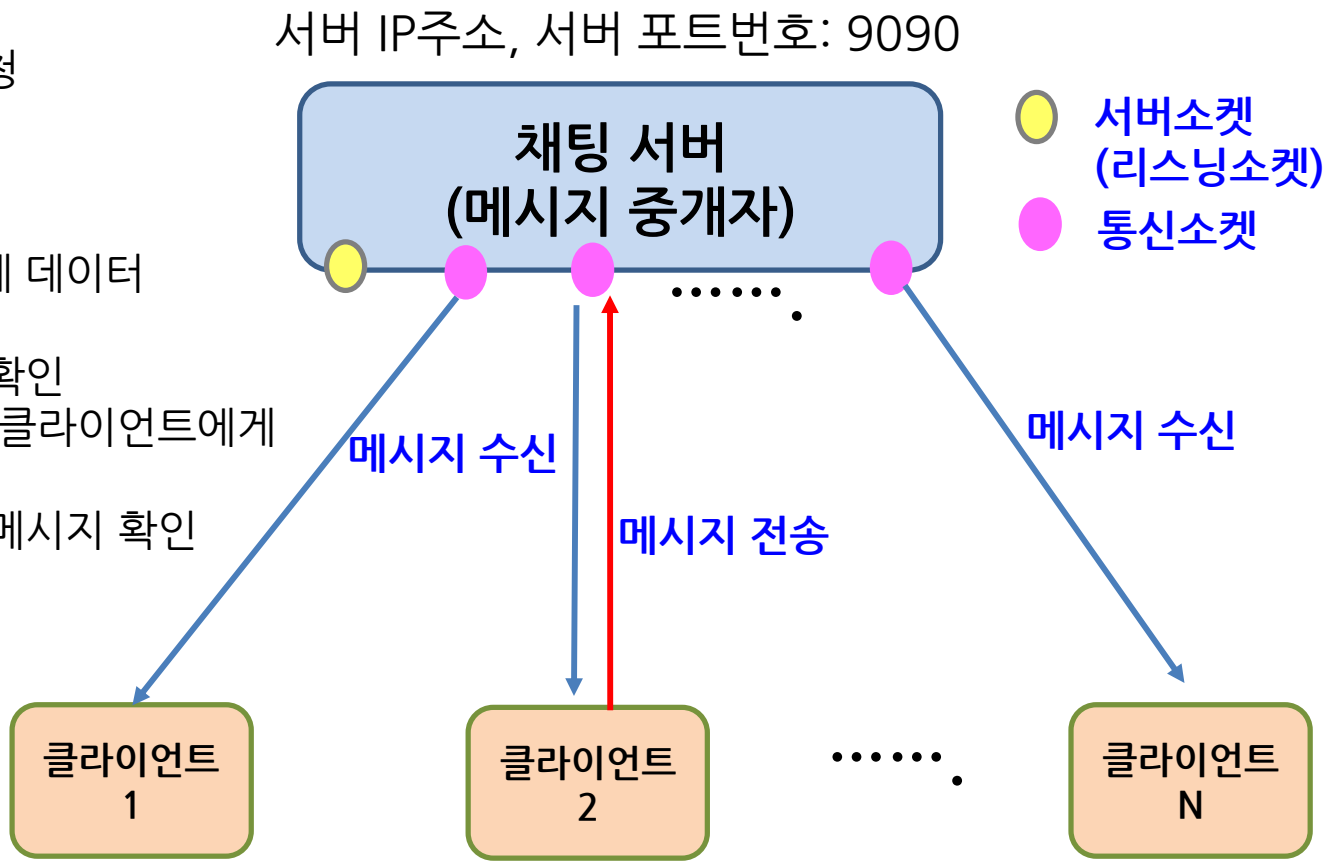
클라이언트 연결요청을 처리하는 부분을 함수로 분리

3. TCP소켓과 스레드를 이용한 그룹채팅

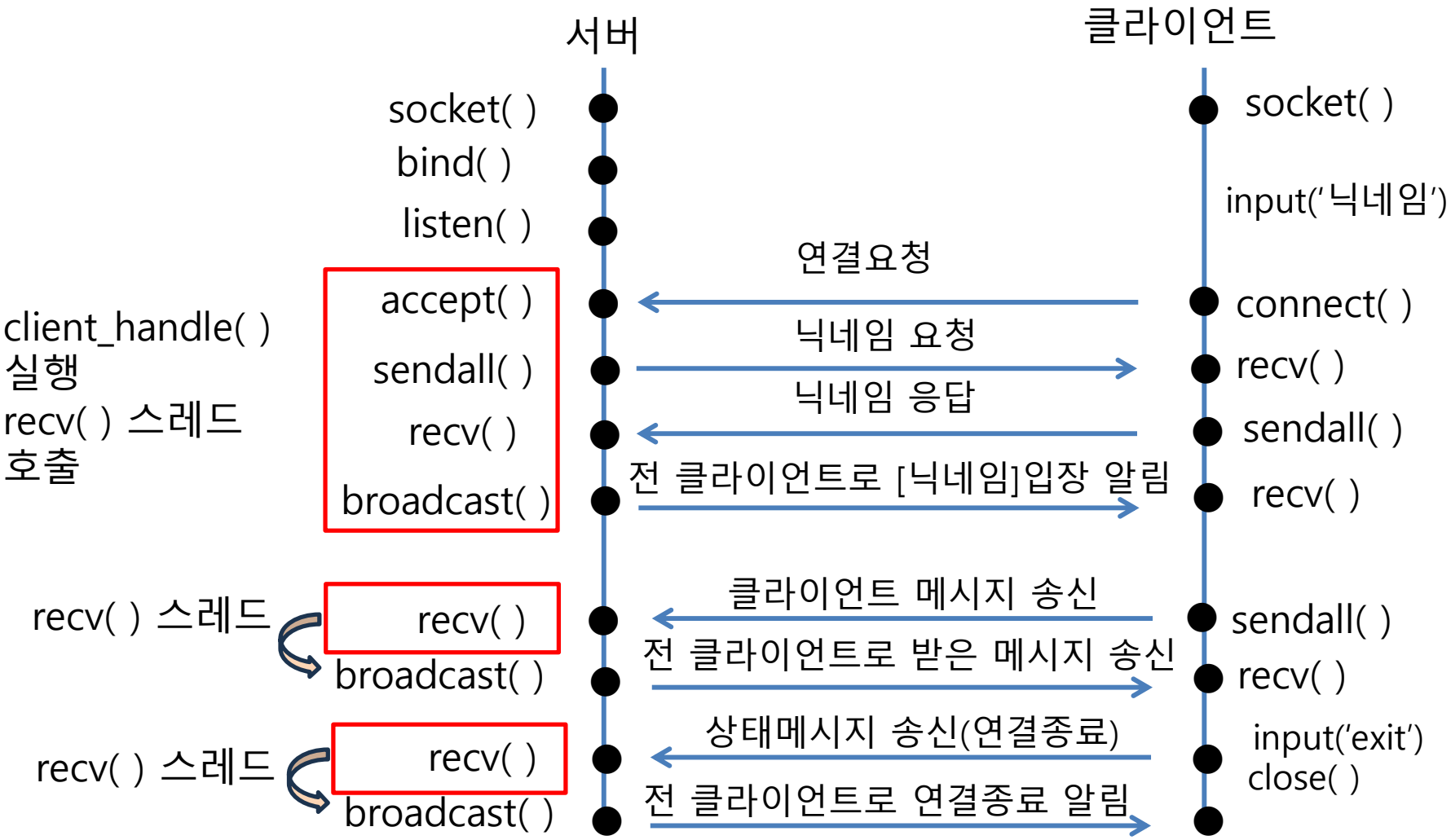
- **TCP 소켓과 스레드를 이용해서 그룹 채팅 프로그래밍**

  - **그룹 채팅방은 이렇게 동작합니다**

1. 채팅 서버 동작
2. 클라이언트 TCP 접속 요청
3. 서버는 클라이언트에게 닉네임 요청
4. 클라이언트 닉네임 전송
5. 특정 클라이언트가 서버에 데이터 전송
6. 서버는 전달받은 메시지 확인
7. 서버에 접속해 있는 모든 클라이언트에게 받은 메시지 전송
8. 모든 클라이언트는 같은 메시지 확인



▪ 그룹 채팅방 메시지 전달 과정



3. TCP소켓과 스레드를 이용한 그룹채팅 - 서버

▪ 그룹 채팅방 서버 프로그램(1)

파일명: tcpGroupChatServer.py

```python
# TCP Group chatting server program w/ thread

import socket, threading

# 클라이언트 소켓과 닉네임을 저장할 리스트 준비
clients = []  # 클라이언트 소켓객체를 저장할 리스트
nicknames = [] # 클라이언트 닉네임을 저장할 리스트

# 메시지 수신되면 대화방에 참여한 전 클라이언트에게 전송
def broadcast(message):
    for client in clients:
        client.sendall(message.encode('utf-8'))

```

```python
# 클라이언트 수신된 메시지 처리
def recv(client):
    while True:
        try: # 정상적으로 메시지 수신되면
             # 수신된 메시지 broadcast로 전달
            rxmsg = client.recv(1024).decode()
            broadcast(rxmsg)
        except: # 네트워크 오류 발생시
            index = clients.index(client)
            clients.remove(client) # 클라이언트 정보삭제
            client.close()   # 클라이언트 소켓 종료
            nickname = nicknames[index]
            broadcast(f"{nickname}님이 퇴장했습니다")
            nicknames.remove(nickname) # 닉네임 삭제
            print(f"{nickname}퇴장, {len(clients)}명 참가
            break

```

▪ 그룹채팅방 서버 프로그램(2)

```
31 # 대화방에 참여하는 클라이언트 관리
32 def client_handle():
33     while True:
34         conn, addr = server.accept()
35         print(f">> {addr} is connected")
36         # 접속된 클라이언트에세 닉네임 요청 및 저장
37         conn.sendall('NICK'.encode('utf-8'))
38         nickname = conn.recv(1024).decode('utf-8')
39         print(f"{nickname} 입장")  # 서버용 출력
40         nicknames.append(nickname)
41         clients.append(conn)
42         # 새로 접속된 클라이언트 정보를 모든 클라이언트에게 알람
43         broadcast(f"{nickname}님이 입장했습니다")
44         broadcast(f"{len(clients)}명이 대화방에 참가중입니다")
45         conn.sendall(f"{nickname}님 반갑습니다".encode('utf-8'))
46         # 새로 참가한 클라이언트의 수신 스레드 생성 및 실행
47         receiver = threading.Thread(target=recv, args=(conn,))
48         receiver.start()
49
```

▪ 그룹채팅방 서버 프로그램(3)

```
53 if __name__ == "__main__":
54     IP = '0.0.0.0'  # 서버 IP
55     PORT = 9090
56     address = (IP, PORT)  # tuple로 주소정보 저장
57
58     print(">> Group Chatting Server start")
59     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
60     # (optional code) winError 10048 에러 해결을 위한 추가 코드
61     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)
62     server.bind(address)
63     server.listen()
64     client_handle()
```

3. TCP소켓과 스레드를 이용한 그룹채팅 - 클라이언트

■ 그룹채팅방 클라이언트 프로그램(1)

파일명: tcpGroupChatClient.py

```
1  # TCP Group chatting client program w/ thread
2  import socket, threading
3
4  def send(client):
5      while True:
6          message = input("Send >>")
7          sendmsg = f"[{nickname}]: {message}"
8          client.sendall(sendmsg.encode('utf-8'))
9          if message == 'exit':
10             print(">> Exit by User")
11             client.close() # 소켓 종료
12             break
13
14 def recv(client):
15     while True:
16         try:
17             # 서버로 수신된 메시지가 'NICK'인 경우 닉네임 전송
18             rxmsg = client.recv(1024).decode('utf-8')
19             if rxmsg == 'NICK':
20                 client.sendall(nickname.encode('utf-8'))
21             else: # 채팅 메시지인 경우 출력
22                 print(rxmsg)
23         except: #네트워크 오류인 경우 소켓종료
24             print("Network error occurred!")
25             client.close()
26             break
```

▪ 그룹채팅방 클라이언트 프로그램(2)

```
27
28 if __name__ == "__main__":
29     IP = '10.2.40.241'  # 서버 IP
30     PORT = 9090         # 서버 포트번호
31     address = (IP, PORT)  # tuple로 주소정보 저장
32
33     # 대화방에 사용할 닉네임 정하기
34     nickname = input("Your nickname:")
35
36     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
37     client.connect(address)
38
39     # 메시지 송신과 수신을 스레드로 생성하고 실행
40     receiver = threading.Thread(target=recv, args=(client,))
41     receiver.start()
42     sender = threading.Thread(target=send, args=(client,))
43     sender.start()
44     while True:
45         receiver.join()
46         sender.join()
47         break
48     print("Client End")
```

■ 그룹채팅방 클라이언트 채팅창

server

```
>> Group Chatting Server start
>> ('117.17.135.85', 50914) is connected
손님1 입장
>> ('117.17.135.85', 50917) is connected
손님2 입장
>> ('117.17.135.85', 50918) is connected
손님3 입장
손님1퇴장, 2명 참가중
```

client1

```
Your nickname: 손님1
손님1님이 입장했습니다
1명이 대화방에 참가중입니다손님1님 반갑습니다
손님2님이 입장했습니다
2명이 대화방에 참가중입니다
손님3님이 입장했습니다
3명이 대화방에 참가중입니다
Send >> 안녕하세요
[손님1]: 안녕하세요
[손님2]: 안녕
[손님3]: 반갑습니다
Send >> 인공지능학과 채팅방입니다
[손님1]: 인공지능학과 채팅방입니다
[손님2]: 모두 반갑습니다
[손님3]: 접속에 성공했습니다
Send >> exit
>> Exit by User
Network error occred!
Client End
```

client2

```
Your nickname: 손님2
손님2님이 입장했습니다
2명이 대화방에 참가중입니다손님2님 반갑습니다
손님3님이 입장했습니다
3명이 대화방에 참가중입니다
[손님1]: 안녕하세요
Send >> 안녕
[손님2]: 안녕
[손님3]: 반갑습니다
[손님1]: 인공지능학과 채팅방입니다
Send >> 모두 반갑습니다
[손님2]: 모두 반갑습니다
[손님3]: 접속에 성공했습니다
[손님1]: exit
손님1님이 퇴장했습니다
Send >>
```

▪ 추가 기능 구현하기

- "서버에서 특정 클라이언트로부터 메시지를 받으면
  메시지를 보낸 클라이언트는 제외하고 모든 클라이언트에게 보내기"
  서버측 코드를 보완해 보세요

```
Your nickname: 손님1
손님1님이 입장했습니다
1명이 대화방에 참가중입니다손님1님 반갑습니다
손님2님이 입장했습니다
2명이 대화방에 참가중입니다
손님3님이 입장했습니다
3명이 대화방에 참가중입니다
Send >> 안녕하세요
[손님3]: 안녕
[손님2]: 반갑습니다
Send >> 네 반가워요
[손님2]: 즐거운 수업
[손님3]: 채팅 성공
Send >>
```

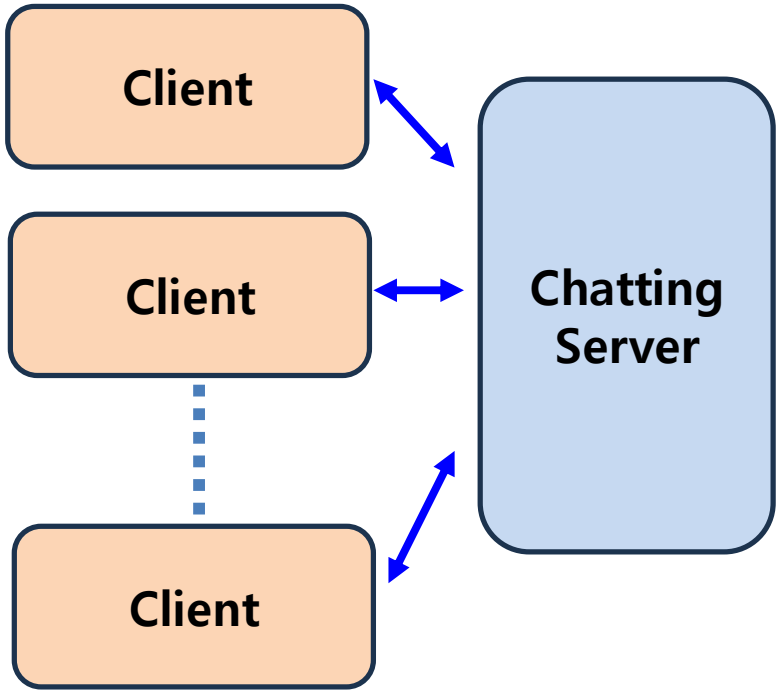
```
Your nickname: 손님2
손님2님이 입장했습니다
2명이 대화방에 참가중입니다손님2님 반갑습니다
손님3님이 입장했습니다
3명이 대화방에 참가중입니다
[손님1]: 안녕하세요
[손님3]: 안녕
Send >> 반갑습니다
[손님1]: 네 반가워요
Send >> 즐거운 수업
[손님3]: 채팅 성공
Send >>
```

클라이언트 채팅창에 자신이 보낸 메시지가 다시 출력되지 않음

4. 응용실습 2

도전과제: 네트워크를 통해 다른 컴퓨터와 그룹채팅방을 통해 채팅하기

- **1개의 채팅서버에 7명 이상 클라이언트가 참가하는 그룹 채팅방 실습하기**
  ***닉네임은 '이름' 사용하기**



1) 한 명이 채팅 서버 담당
2) 다른 사람은 클라이언트로 참가
   단, 총 클라이언트 수 7명 이상 참가
4) 클라이언트 입장 확인
5) 클라이언트간 채팅 확인
6) 클라이언트 종료 확인

* 채팅 시 자신이 보낸 메시지는 채팅 화면에 출력되지 않아야 함 (추가기능 확인)

5주차 실습 과제

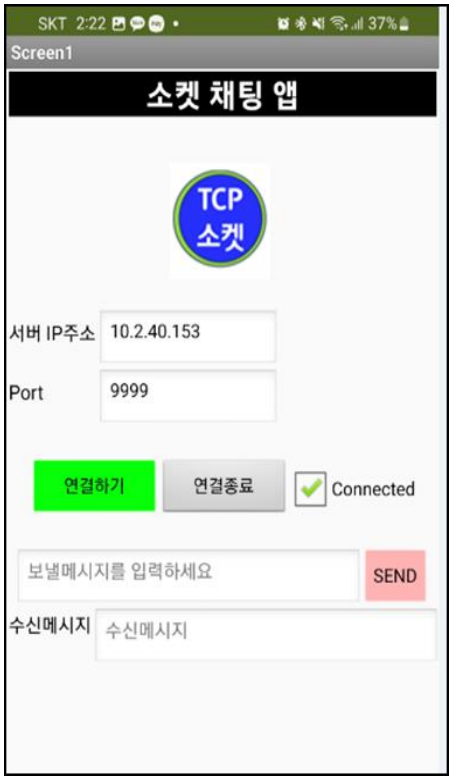
- **과제명: 스레드와 TCP 소켓을 이용한 그룹 채팅**

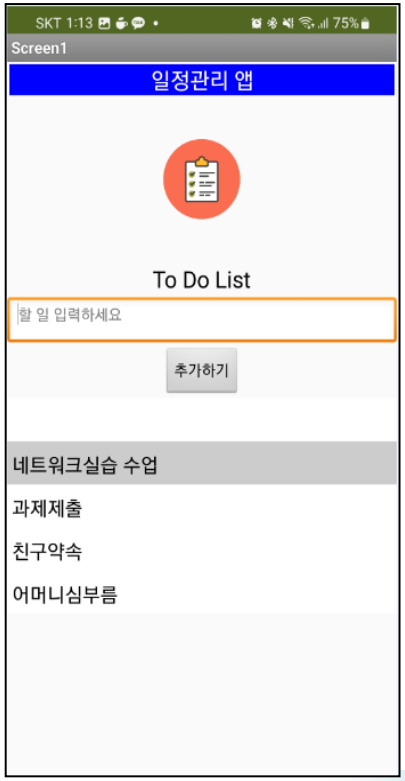
  - 제출방법: 5주차 실습한 그룹채팅 최종 파이썬 파일을 PC에 저장 후
    과제 제출란에 제출
  - 첨부파일: **tcpGroupChatServer_이름.py , tcpGroupChatClinet_이름.py**
  - 프로그램 상 오류 혹은 이해가 안 되는 부분은 comment 남겨 주세요.

6주차 영상수업 안내

■ 6주차 수업은 임시 이러닝 수업

- 학습방법: CTL 사이트 6주차 영상 수업 (임시 이러닝)
- 학습기간: 4/13(월) ~ 4/12(일)
- 주제: 앱 인벤터를 이용한 앱 제작 실습
- 과제: 제작한 앱의 프로젝트 파일을 학습기간내 과제로 제출





Thank You